

Computer Systems Architecture

JAMES ZHANG*

September 25, 2024

These are my notes for UMD's MATH411: Computer Systems Architecture. These notes are taken live in class ("live- $\text{\LaTeX}$ -ed). This course is taught by Professor Ilchul Yoon, whose notes are inspired by *Computer Architecture: a Quantitative Approach* by Hennessy and Patterson.

Contents

1	Computer Architecture: Introduction	2
2	Instruction Set Architectures	3
2.1	Operations in the Instruction Set	4
2.2	Types and Sizes of Operands	4
2.3	Addressing Modes	4
2.4	Addressing Memory	5
2.5	Encoding and Instruction Formats	5
3	Performance Metrics	5
4	Summarizing Performance, Amdahl's Law, and Benchmarks	7
5	Execution of a Complete Instruction	9
5.1	Datapath Implementation	9
5.2	Control Flow Implementation	11
6	Pipelining - MIPS Implementation	11
7	Pipeline Hazards	14
8	Handling Data Hazards	16
9	Handling Control Hazards	17
10	Midterm 1 Preparation	19
10.1	Practice Set 1	19
10.2	Practice Set 2	24

*Email: jzhang72@terpmail.umd.edu

§1 Computer Architecture: Introduction

Definition 1.1 (Computer). A computer is a sophisticated electronic calculating machine that accepts input information, processes the information according to a list of stored instructions, and finally produces resulting output information. From a high level, the computer has the following components

- Input component that takes in information
- Processing unit that processes the information
- Memory unit that stores the information
- Output unit that outputs the data

Definition 1.2 (Central Processing Unit). Combined data path which is where your information flows and control path which coordinates the activities of the various units of the computer system. Together, these form the CPU.

Definition 1.3 (Instruction Set Architectures). ISAs give a logical view of what a computer is capable of doing and when you look at computer organization, it talks about how ISAs are implemented. Together, ISA and computer organization comprise computer architecture.

Definition 1.4. Instructions are basic commands like transferring information from one register to another register. Instructions can control the flow of the program, think about jumping in MIPS. A sequence of instructions is called a program, which is stored in memory.

Example 1.5

Suppose you wanted to add two numbers, and those numbers are stored in memory.

Process. From memory, you have to bring the numbers to the adder unit and add. Need data transfer instructions to transfer the data from memory to the processor, and an add instruction to add. □

Definition 1.6 (Data). These instructions operate on decimal numbers, imaginary numbers, octal numbers, or encoded characters. The memory unit stores instructions as well as data as a sequence of bits.

Definition 1.7 (Memory Access). The number of memory accesses depends on the machine, but each unique address is 10 bits, and note that we have a hierarchical memory system. We have a small amount of RAM with multiple levels of caches, and then we have the main memory and secondary storage.

Definition 1.8 (Bus). A bus is an interconnection of wires, capable of carrying bits of wires to the different components of a digital computer. Functional units are connected by means of a group of parallel wires. When you talk about a processor

with a word length of 64-bits, the processor operates on 64 bits of data, and so it is reasonable that the bus can transfer 64 bits of data from one point of the computer to another point.

Definition 1.9 (von Neumann Architecture). The von Neumann Architecture, also known as a stored-program computer changed history - treating instructions in the same way as data $\implies$ programs can be simply shipped as files of binary numbers that maintain binary compatibility and computers can inherit ready-made software that are compatible with the existing ISA.

§2 Instruction Set Architectures

Remark 2.1. MIPS is an example ISA! Recall that computer architecture consists of instruction set architecture and the computer organization itself. The ISA specifies what the processor is capable of doing $\implies$ interface between the hardware and software, so the only way to interact with the hardware is through the ISA.

Note 2.2. Suppose you have a high-level program language written in C, this code needs to be translated to assembly language and its instructions, and then this has to be translated to binary code that the processor understands. The compiler and assembler are responsible for this translation.

Definition 2.3 (Taxonomy of ISAs). ISAs differ based on the internal storage in a processor ie. where the operands are stored, and whether they are named explicitly or implicitly.

- **Single accumulator organization** names one of the general purpose registers as the accumulator, so one of the operands is the accumulator and it is enough if the other operand is specified along with the instructions
- **General register organization** specifies all operands explicitly, further classification depends on whether the operands are available in memory or registers.
 - **Register-register**, where registers are used for storing operands. These are also called **Load-store** architecture.
 - **Register-memory**, where one operand is in register and the other is in memory
 - **Memory-memory**, where all operands are specified as memory operands
- **Stack organization**, where the operands are in a stack, and operations are carried out on the top of stack, so this method is implicit.

Now, we will discuss the different features that must be considered when designing the ISA.

§2.1 Operations in the Instruction Set

Note 2.4. A computer must have the following types of instructions: data transfer, data manipulation, program sequencing, control, and IO instructions.

Definition 2.5 (Data Transfer Instructions). Data transfer instructions perform data transfer between the various storage places in the computer system. Since both instructions and data are stored in memory, the processor needs to be able to read both from memory. Logically, we get the two following instructions

- **Load** (or Read or Fetch) - transfers a copy of the data from the memory to the processor
- **Store** (or Write) - moves data from the processor to memory

Other instructions are needed to transfer data from one register to another or from other devices to the processor

Definition 2.6 (Data Manipulation Instructions). These instructions contain arithmetic operations, logical operations, or shift operations.

Definition 2.7 (Program Sequencing and Control Instructions). Since a program is a sequence of instructions, sometimes we want to use control instructions to change the flow of the program $\Rightarrow$ **branch instructions**. The new address is called the **branch target**. The branch instruction can be conditional or unconditional. If the condition is not satisfied, then the program counter is incremented in a normal way.

Definition 2.8 (Input and Output Instructions). Used for transferring information between registers, memory and I/O devices.

§2.2 Types and Sizes of Operands

Definition 2.9 (Data Types and Sizes). Indicate the various data types supported by the processor and their lengths. Common operand types are: characters (8 bits), half words (16 bits), words (32 bits), single precision floating point (1 word), double precision floating point (2 words), integers

Two's complement binary numbers, characteristics in ASCII, floating point numbers following the IEEE Standard 754 Packed and unpacked decimal numbers.

§2.3 Addressing Modes

Definition 2.10 (Addressing Modes). The way the operands are chosen during program execution is dependent on the **addressing mode** of the instruction. The addressing mode specifies a rule for interpreting or modifying the address field of the instruction before the operand is actually referenced. There is

- Register mode
- Absolute (direct) mode
- Immediate mode

- Indirect mode
- Index mode
- Relative mode

§2.4 Addressing Memory

Definition 2.11 (Autoincrement and Decrement Modes). The following two does are useful for accessing data items in successive locations in the memory. There is autoincrement mode and autodecrement mode.

§2.5 Encoding and Instruction Formats

Note 2.12. We have discussed how the processor can execute different types of instructions and the different ways to specify operands — now the information must be given to the processor in the form of an **instruction format**, where the number of bits in the instruction is divided into groups called **fields**.

Definition 2.13. The most common fields found in instruction formats are

- An operation code field that specifies the operation to be performed
- An address field that designates a memory address or processor register
- A mode field that specifies the way the operand or effective address is handled

§3 Performance Metrics

In this section, we will discuss the performance metrics for a processor and also discuss the CPU performance equation.

Definition 3.1 (Execution Time). **Execution time** is the time between the start and the completion of a task that is important to individual users. This does not include the time waiting for IO or running other programs.

Example 3.2

- Unix **time** command example:
 - 90.7u 12.9s 2:39 65%
 - The user used the CPU for 90.7 seconds (user CPU time)
 - The system used it for 12.9 seconds (system CPU time)
 - Elapsed time (wall clock time) from the user's request to completion of the task was 2 minutes, 39 seconds (159 seconds)
 - And $(90.7 + 12.9)/159 = 65\%$
 - » the rest of the time was spent waiting for I/O or running other programs

Definition 3.3 (Throughput). Throughput is the total amount of work done in a given time is important to data center managers $\implies$ servers

Definition 3.4 (Dependability). Infrastructure providers started to offer **service level agreements** to guarantee how dependable their service will be. Failure is a transition from working to not working, and restoration is a transition from not working to working.

Definition 3.5 (Dependability Metrics).

- **Mean time to Failure (MTTF)** measures reliability, and **Failures in Time (FIT)** $= \frac{1}{\text{MTTF}}$, reported as failures per billion hours.
- **Mean time to repair (MTTR)** measures service interruption
- **Mean time between Failures (MTBF)** $= \text{MTTF} + \text{MTTR}$
- **Module Availability** $= \text{MTTF} / \text{MTBF} = \text{MTTF} / (\text{MTTF} + \text{MTTR})$

Note 3.6. If modules have exponentially distributed lifetimes, the age of the module does not affect its probability of failure, the overall failure rate (FIT) is the sum of failure rates of the modules.

Definition 3.7 (Performance). Performance is the inverse of execution time

$$\text{Performance} = \frac{1}{\text{Execution Time}}$$

Note 3.8. Decreasing response time almost always improves throughput.

Definition 3.9 (CPU Execution Time for a Program). To determine the CPU execution time for a program, you can find out the total number of clock cycles that the program takes and multiply it by the clock cycle time

$$\text{Execution Time for Program} = \text{Clock Cycles for Program} \times \text{Clock Cycle Time}$$

$$\text{Clock Cycle Time} = \frac{1}{\text{Clock Rate}}$$

$$\text{Execution Time for Program} = \frac{\text{Clock Cycles for Program}}{\text{Clock Rate}}$$

To improve execution time for a program, you can improve performance by either reducing the length of clock cycle time or by the number of clock cycles required for a program.

Note 3.10. Not all instructions take the same number of clock cycles.

$$\text{CPU Time} = \text{Instruction Count} \times \text{CPI} \times \text{Clock Cycle}$$

where **CPI** is cycles per instruction.

§4 Summarizing Performance, Amdahl's Law, and Benchmarks

Remark 4.1. In the previous section, we learned that CPU Execution Time is the most consistent measure of performance, but what happens when processors execute multiple programs. How do you summarize performances and compare them?

Definition 4.2 (Summarizing Performance). To get a single number which summarizes performance, we can take the arithmetic mean, or weighted arithmetic mean, but most people use a **geometric mean**. One of the most popular benchmarks is the **Standard Performance Evaluation Corporation (SPEC)** benchmarks. Rather than pick weights and get weird results, we could normalize execution times to a reference computer by dividing the time on the reference computer by the time on the computer being rated, yielding a ratio proportional to performance.

Example 4.3

Suppose that the SPEC of computer A on a benchmark was 1.25 times higher than computer B, then you would know

$$\begin{aligned} 1.25 &= \frac{SPEC_A}{SPEC_B} = \frac{\text{ExecutionTime}_{Ref}}{\text{ExecutionTime}_A} / \frac{\text{ExecutionTime}_{Ref}}{\text{ExecutionTime}_B} \\ &= \frac{\text{ExecutionTime}_B}{\text{ExecutionTime}_A} = \frac{\text{Performance}_A}{\text{Performance}_B} \end{aligned}$$

Note that the SPEC Ratios are unitless.

Definition 4.4 (Geometric Mean). Using the geometric mean ensures two important properties

1. Geometric mean of the ratios is the same as the ratios of the geometric means
2. The ratio of the geometric means is equal to the geometric mean of the performance ratios, which implies that the choice of the reference computer is irrelevant.

$$\text{GeometricMean} = \sqrt[N]{\prod_{i=1}^N \text{SPECRatio}_i}$$

Note 4.5. Some important observations about designs of computer designs to optimize performance

- Take advantage of parallelism through multiple processors, having multiple threads of execution, exploiting ILP through pipelining
- **The locality of reference** rule-of-thumb says that a program spends 90% of its time in 10% of its code, so we should put effort into taking advantage of this through **memory hierarchy**. **Temporal** locality states that recently

accessed items are likely to be accessed in the near future. **Spatial** locality says that items whose addresses are near one another tend to be referenced close together in time.

- Focus on the common case $\implies$ make addition faster, instead of optimizing the square root.

Theorem 4.6 (Amdahl's Law)

Amdahl's Law quantifies how much of a speedup one could get for a given enhancement.

Speedup is the time the task took originally divided by the time the task takes after improvement. Suppose the original task takes 1 second, f seconds in the critical piece and $1 - f$ in the non-critical. Then the improved task will only take f/s in the critical piece, but still $1 - f$ in the noncritical. Therefore, the

$$\text{Speedup} = \frac{\text{Old Time}}{\text{New Time}} = \frac{1}{(1 - f) + \frac{f}{s}}$$

Here s is the speedup factor, note that we want to target operations that have large f to maximize speedup.

Example 4.7

Suppose we get a speedup of 2 after we replace the GPU card. If the new card can process GPU instructions 5 times faster, what is the percentage of instructions affected by the new GPU?

Solution. We want to find f and $s = 5$

$$2 = \frac{1}{(1 - f) + \frac{f}{5}} \implies 2 - 2f + \frac{2f}{5} = 1 \implies \frac{-8f}{5} = -1 \implies f = \frac{5}{8} = 0.625$$

□

Note 4.8. The performance of any system is constrained by the speed or capacity of the slowest point.

Note 4.9. The impact of an effort to improve the performance of a program is primarily constrained by the amount of time that the program spends in parts of the program not targeted by the effort

Example 4.10

Suppose that for the same cost, we can speed up integer arithmetic by a factor of 20 or speed up floating point arithmetic by 2. If our task spends 10% of its time in integer arithmetic, and 40% of its time in floating point arithmetic, what should we do?

Solution.

$$\text{Option 1 Speedup} = \frac{1}{0.9 + \frac{0.1}{20}} \approx 1.105$$

$$\text{Option 2 Speedup} = \frac{1}{0.6 + \frac{0.4}{2}} \approx 1.25$$

so we should choose option 2 and do the floating point arithmetic speedup. $\square$

§5 Execution of a Complete Instruction

Definition 5.1 (Pseudoinstructions). **Pseudoinstructions** do not have hardware implementations. Instead, they are mapped to multiple basic instructions, and these are convenient for programmers. For example,

`li R16, 10  $\implies$  ori R16, R0, 10`

Some other examples of pseudoinstructions are `b`, `beqz`, `bge`, `bgeu`, `bgt`, `bgtu`, `blt`, `bltu`.

Note 5.2 (Data Path vs. Control). **Data path** has storage functional units, and interconnections sufficient to perform the desired functions. Inputs are control points, and outputs are signals. On the other hand, **controller** has state machine to orchestrate operation on the data path based on desired function and signals.

§5.1 Datapath Implementation

The objective of this section is to discuss how an instruction gets executed in a processor and the datapath implementation, using the MIPS architecture as a case study.

Note 5.3 (MIPS Characteristics). Some details regarding the MIPS ISA

- Load/store only displacement addressing
- 32 general purpose registers (GPRs) with register R0 always having 0
- 32 floating point registers (FPRs)
- No floating point status register (FP status register)
- No condition codes
- Supported addressing modes are immediate, displacement, and register mode

- 3 fixed length formats are supported: *I*-type, *R*-type, and *J*-type.

We will examine the MIPS implementation for a simple subset that shows most aspects of implementation

- Memory-reference instructions: `lw`, `sw`
- Arithmetic-logical instructions: `add`, `sub`, `and`, `or`, `slt`
- Branch instructions: `beq`, `j`

While we are missing many instructions, we will illustrate the key principles in creating a datapath and designing the control.

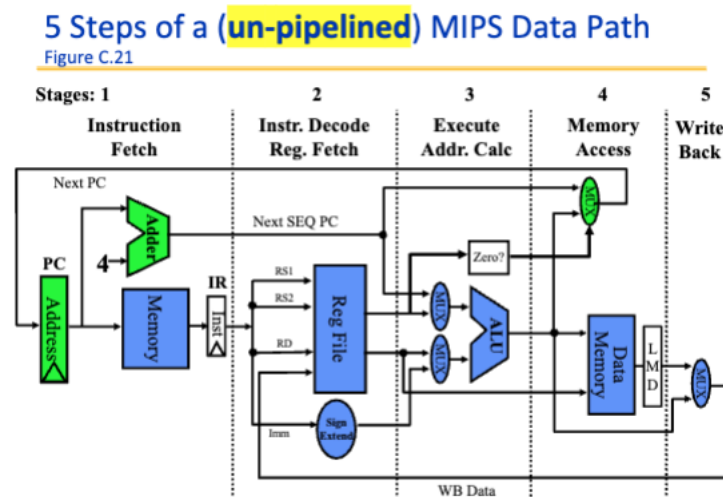
Note 5.4. When we look at the instruction cycle of any processor, it should involve the following operations

1. Fetch instruction from memory
2. Decode the instructions
3. Fetch the operands from memory
4. Execute the instruction
5. Write or store the result

Note 5.5. A lot of these steps for the subset of instructions is the same, independent of the exact class of the instruction. For every instruction, the first two steps of instruction (fetch, decode) are identical.

- Send PC to the program memory that contains the code and fetch the instruction
- Read one or two registers. `lw` reads only one register, but most read two. Since MIPS uses fixed length format with register specifiers in the same place, the registers can be read, irrespective of the instruction

Next, most instructions use ALU. For example, load/store use it for effective memory address calculation. Then, access the memory. `lw` will read and then write the data fetched from memory to a register.



We also need to implement the necessary control signals. The third multiplexor determines whether $PC + 4$ or the branch destination is written into the PC .

§5.2 Control Flow Implementation

not sure if this section is needed

⋮

Definition 5.6 (Single-Cycle Implementations). **Single-Cycle Implementations** are sequential, no parallelism, and one (long) cycle for processing each instruction. It is not feasible to vary the period of different instructions. Furthermore since the clock cycle is equal to the worst case delay, this violates the design principle of making the common case fast. This implementation is inefficient in performance and hardware cost.

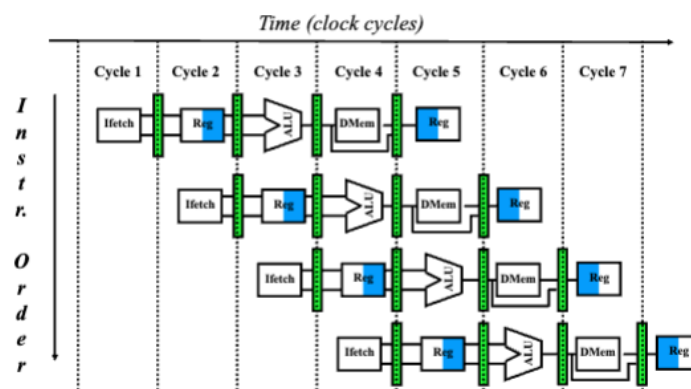
Definition 5.7 (Multi-Cycle Implementations). **Multi-cycle implementations** are sequential, no parallelism and processing instructions in multiple separate stages. 1 cycle for each stage, and clock cycle time still has to be set to the time for the longest stage.

§6 Pipelining - MIPS Implementation

Remark 6.1. Pipelining is analagous to an apartment of college students doing laundry. If someone puts their load in when someone else moves their clothes from the washer to the dryer, they can significantly speed up washing time.

Visualizing Pipelining

Figure C.3



To apply the concept of instruction execution in pipeline, it is required to break the instruction execution into different tasks. Each task will be executing in different processing elements of the CPU. We can split the process up into two: instruction fetch and instruction execution.

Definition 6.2 (Cycle Time of an Instruction Pipeline). Cycle time τ can be determined as

$$\tau = \max[\tau_i] + d = \tau_m + d, \quad i \in [1, k]$$

where τ_m is the maximum stage delay, k = the number of stages in the instruction pipeline, and d = the time delay needed to advance signals and data from one stage to the next. The non-pipelining example would take time

$$T_k = [k + (n - 1)]r$$

On the other hand, let instruction execution be divided into five stages: fetch, decode, execute, memory access, and write back which we will denote as **Fi**, **Di**, **Ei**, **Mi**, and **Wi**. When the pipeline is filled, there are five different activities taking place in parallel.

- ie. when the first instruction's decode happens, the second instruction's fetch is done
- $\implies$ 5 instructions are in progress at once $\implies$ 5 distinct hardware units are needed, and they must be able to perform their task independently

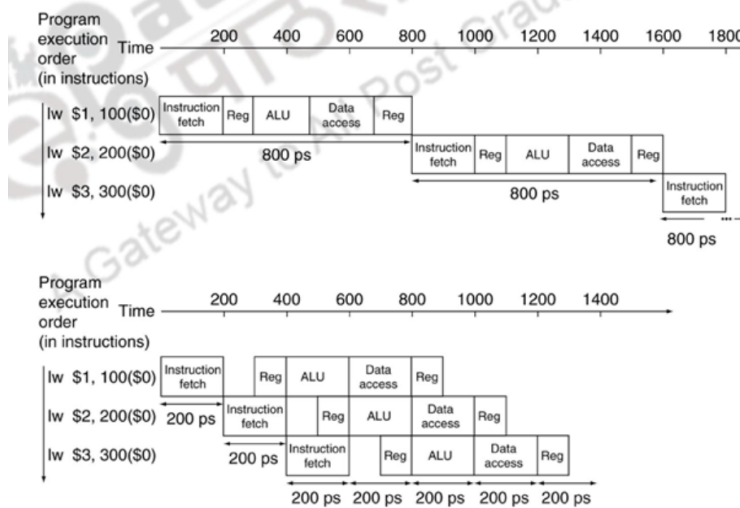
Note 6.3 (Pipelining Speedup). The speedup from pipelining is due to increased throughput and the latency (time for each instruction) does not decrease.

Note 6.4 (Features of Pipelining). The basic features of pipelining are

- Pipelining does not help latency of a single task, only helps increase throughput of entire workload
- Pipeline rate is limited by the slowest pipeline stage
- Multiple tasks operate simultaneously
- Exploits parallelism among instructions in a sequential instruction stream
- Unbalanced lengths of pipe stages reduces speedup
- Time to "fill" pipeline and "drain" it reduces speedup

Note 6.5 (Five Stages MIPS Pipeline). Consider the MIPS Pipeline with five stages, with one step per stage

- **IF**: Instruction fetch from memory
- **ID**: Instruction decode and register read
- **EX**: Execute operation or calculate address
- **MEM**: Access memory operand
- **WB**: Write result back to register



Note that for a non-pipelined implementation, it takes $800ps$ for each instruction, and for the pipelined implementation, it takes $200ps$.

Note 6.6. If all stages are balanced in pipelining ie. they all take the same time, then

$$\text{Time Between Instructions}_{\text{Pipeline}} = \frac{\text{Time Between Instructions}_{\text{Nonpipelined}}}{\text{Number of Stages}}$$

and so the potential speedup is equal to the number of pipe stages.

Definition 6.7 (Buffers in Pipelining). **Buffers** are mandatory in pipelining — each stages takes in data from that buffer, processes it, and writes it into the next buffer.

- **Buffer IF/ID** holds fetched instruction I4
- **Buffer ID/EX** holds decoded instructions and source operands for I3
- **Buffer EX/MEM** holds the executed results of I2
- **Buffer MEM/WB** holds data fetched from memory (for a load) for I1, and for the arithmetic and logical operations

Since there are five stages, it makes sense that there are four buffers $\implies$ there need not be a buffer for the IF and WB stages to interact.

Note 6.8. Note that the entire information of the instruction is written into each of the buffers.

Example 6.9

Consider the following example which illustrates this further

- If the PC fetches the instruction, it is written into the IF/ID buffer and the PC is incremented by 4
- Next, the instruction is decoded, the register file is read, and the operands are written into the ID/EX buffer
- The base register's contents and sign are fed to the ALU, which calculates the memory address, and this is stored in the EX/MEM buffer. Also the destination register info is moved from the ID/EX buffer to EX/MEM buffer.
- Next, the memory access happens and the read data is written into the MEM/WB buffer, and the destination register's info is passed from EX/MEM to MEM/WB, too.

§7 Pipeline Hazards

Definition 7.1 (Hazards). Ideally, we expect a CPI value of 1 when leveraging pipelining, but there are some factors that limit this called **hazards**, which arise in the pipeline and prevent the next instruction from executing during its designated clock cycle. There are three types of hazards:

- **Structural hazards**: when two instructions require the same resource.
- **Data hazards**: Instruction depends on result of prior instruction that is still in the pipeline.
- **Control hazards**: caused by delay between fetching the instructions and decisions about changes in control flow (branches and jumps)

Note 7.2 (Resolving Structural Hazards). To resolve structural hazards, one can wait or throw more hardware at the problem

Note 7.3. Example: If an instruction needs to access memory for an instruction fetch and another instruction is accessing memory for data/access (load/store), these two are conflicting and gives rise to a hazard.

The solution is dual memory pipelines, as opposed to single memory.

Definition 7.4 (Types of Dependencies). There are two types of data dependence:

- **True data dependencies**: an instruction j is **data dependent** on instruction i if either of the following holds
 - Instruction i produces a result that may be used by instruction j
 - Instruction j is data dependent on instruction k , and instruction k is data dependent on instruction i

- **Name dependence**: when two instructions use the same register or memory location, called a **name**, but there is no flow of data between instructions associated with that name
 - **Antidependence**: when instruction j writes a register or memory location that instruction i reads.
 - **Output dependence**: occurs when i and j write the same register or memory location $\implies$ register renaming can fix this and can be done either statically by the compiler or dynamically by the hardware.

Definition 7.5 (Control Dependencies). There are two general constraints imposed by control dependencies:

- An instruction that is control dependent on the branch cannot be moved before the branch so that its execution is no longer controlled by the branch.
- An instruction that is not control dependent on the branch cannot be moved after the branch so that its execution is now controlled by the branch.

Definition 7.6 (Read After Write (RAW)). **RAW** corresponds to a true data dependence. Program order must be preserved, and this hazard results from an actual need for communication $\implies j$ reads data before i writes it.

Definition 7.7 (Write After Write (WAW)). **WAW** corresponds to an output dependence, specifically, it's the same name that causes the issue. This occurs when there are multiple writes. j should write after i has written the data. The solution is register renaming, or just using some other register.

Definition 7.8 (Write After Read (WAR)). **WAR** arises from anti-dependence, and occurs when there are early writes *and* late reads, or when instructions are re-ordered. Again, no data transfer, it is the same name that causes the problem. j should write after instruction i has read the data. The solution is register renaming, similar to WAW.

Note 7.9. MIPS pipelines with a single memory stage face issues because load/store and instruction fetches both require data access $\implies$ pipelined datapaths require separate data memories.

Note 7.10. RAW hazards can happen in any architecture, but WAR and WAR can't happen in MIPS 5 stage pipelines.

Theorem 7.11 (Speedup Equation with Stalls)

$$CPI_{\text{Pipeline}} = \text{Ideal CPI} + \text{Average Stall Cycles per Inst}$$

$$\text{Speedup} = \frac{\text{Ideal CPI} \times \text{Pipeline Depth}}{\text{Ideal CPI} + \text{Pipeline Stall CPI}} \times \frac{\text{Cycle Time}_{\text{Unpipelined}}}{\text{Cycle Time}_{\text{Pipelined}}}$$

$$\text{Speedup} = \frac{\text{Pipeline Depth}}{1 + \text{Pipeline Stall CPI}} \times \frac{\text{Cycle Time}_{\text{Unpipelined}}}{\text{Cycle Time}_{\text{Pipelined}}}$$

Assuming all stages are perfectly balanced and ignoring cycle time overhead, then

$$\text{Speedup} = \frac{\text{Pipeline Depth}}{1 + \text{Pipeline Stall CPI}}$$

Example 7.12

Suppose Machine *A* has dual ported memory and machine *B* has single ported memory, but its pipelined implementation has a 1.05 times faster clock rate. Note that the ideal $CPI = 1$ for both. Assume that loads are 20% of instructions executed.

Solution.

$$\text{Speedup}_A = \frac{\text{Pipeline Depth}}{1 + 0} \times \frac{\text{Cycle Time}_{\text{Unpipelined}}}{\text{Cycle Time}_{\text{Pipelined}}}$$

$$\text{Speedup}_B = \frac{\text{Pipeline Depth}}{1 + 0.2 \times 1} \times \frac{\text{Cycle Time}_{\text{Unpipelined}}}{\text{Cycle Time}_{\text{Pipelined}}/1.05}$$

$$\frac{\text{Speedup}_A}{\text{Speedup}_B} = \frac{1.2}{1.05} = 1.14$$

so machine *A* is 1.14 times faster! □

§8 Handling Data Hazards

In this section, we will discuss how data hazards are handled in general and in the MIPS architecture.

Definition 8.1 (Forwarding). **Forwarding** is the concept of making data available to the input of the ALU for later instructions *before* the generating instruction has gotten to WB in order to write the memory or registers. This allows other instructions to read the data before it's fully been written. This is also called **short circuiting** or **by passing**.

Note 8.2. One way of avoiding/minimizing stalls due to true data dependencies is to reorder the code — separate the dependent instructions. For example,

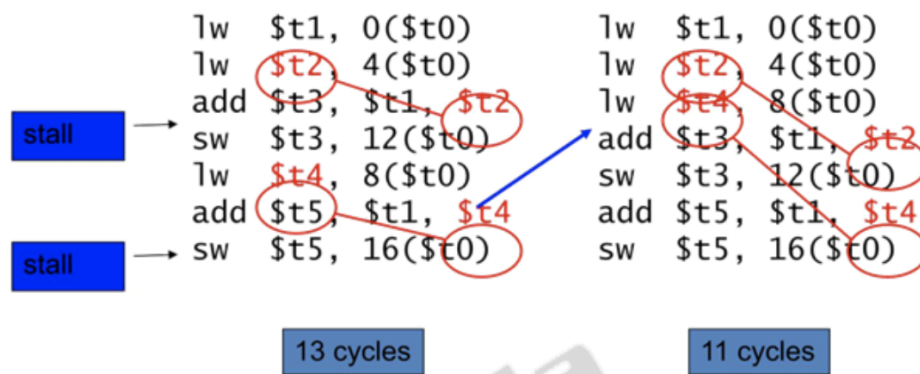


Figure 12.5

Note 8.3. There are instances where stalls may occur even with forwarding.

§9 Handling Control Hazards

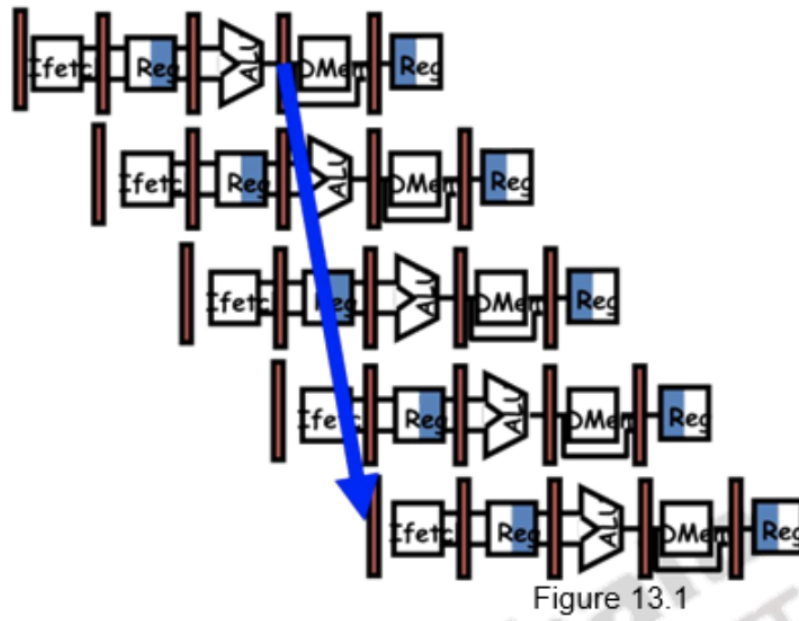
The objective of this section is to discuss how to handle control hazards, to differentiate between static and dynamic branch prediction, and to study the concept of delayed branching.

Definition 9.1 (Branch (Control) Hazard). An instruction must be fetched every clock cycle to sustain the pipeline. However, the delay in determining the proper instruction to fetch the next instruction is called a **control hazard**.

Note 9.2. The two major issues to control dependencies are

- **Exception Behavior:**
- **Handling and Preservation of Data Flow:**

Note 9.3. In MIPS pipeline, the comparison of registers and target address calculation is usually known by the end of the decode stage, which causes a three clock cycle penalty.



Note that if we added hardware to resolve the branch in the decoding stage, the penalty could be reduced. Resolving the branch earlier requires two actions to occur:

1. Computing branch target address
2. Evaluating branch decision earlier

Note 9.4. There are two ways of handling control hazards:

1. Stall until the branch outcome is known or perform the fetch again $\implies$ too much penalty. Not handling these brings down performance and we are violating the principal of making "common cases fast"
2. Predict the behavior of branches
 - Static prediction by the compiler
 - Dynamic prediction by the hardware

Note 9.5. There are primarily three methods adopted:

1. **Predict not taken:** assume that the branch is not taken ie. the condition will not evaluate to true. Recall that the registers are read during ID. Load in the next instruction $PC + 4$ and continue. If we are wrong, the pipeline is cleared of any code taken from the "not-taken" path, and execution continues.
2. **Predict taken approach:** assume that the branch is taken ie. the condition will evaluate to true, and assume that the branch is always taken. Doesn't work for a MIPS architecture with a 5-stage pipeline.
3. **Delayed branching:** a more effective solution

§10 Midterm 1 Preparation

§10.1 Practice Set 1

Example 10.1

What is the speedup with N processors if 80% of the application is parallelizable, ignoring the cost of communication?

Solution.

$$s = \frac{1}{0.2 + \frac{0.8}{N}}$$

□

(b) What is the speedup with 8 processors if, for every processor added, the communication overhead is 0.5% of the **original** execution time. (Assume that 80% of the application is parallelizable, and also that the communication overhead increases linearly as more processor is added.)

Solution.

$$s = \frac{1}{0.2 + 8 * 0.005 + \frac{0.8}{N}}$$

□

Example 10.2

Suppose that we have a single processor with an FIT of 150. What is the MTTF for this processor in (hours)?

Solution.

$$FIT = \frac{10^9}{MTTF} \implies MTTF = \frac{10^9}{150} = 6,666,667 \text{ hours}$$

□

(b) Assume that a 100-processor system is built using the processor above, and also assume that it takes 30 days to get the system running again once the system crashes. What is the availability of the system?

Solution.

$$\text{Availability} = \frac{MTTF}{MTTF + MTTR} = \frac{6,666,667}{6,666,667 + 24 * 30} \approx 0.99$$

□

Q3. Instruction Set Architecture

Consider the following fragment of C code:

```
if (A > B) {
    C = -(A+B);
}
```

Consider the following constraints to answer this question.

- Assume that A, B, and C are 64-bit integers.
- Use instructions for 64-bit integers – e.g., use DADD (for 64-bit) instead of ADD (for 32-bit). Similarly, use LD/SD instead of LW/SW (for 32-bit). LD stands for *Load Double Word*.
- All data values are kept in memory (at addresses 2000, 2008, and 2016 for A, B, and C, respectively). Assume that 4, 8, and 10 are stored in the memory addresses 2000, 2008, and 2016, respectively.
- You must use BNE or BEQ instruction for the branching. (Hint: See SLT (Set-on-Less-Than) instruction. For MIPS references, check the course homepage.)

In the next page, write your answers for the following questions:

- Write the code in MIPS assembly.
- How many instructions are executed **dynamically**?
- How many memory-data references will be executed?
- What is the code size in bytes? (Count only the bytes for instructions.)

Example 10.3

Let's begin with the code.

```
1  # Load A.
2  LD $t1, 2000($0)
3  # Load B.
4  LD $t2, 2008($0)
5  # Sets $t3 to 1 if B < A else 0.
6  SLT $t3, $t2, $t1
7  # Branch: if $t3 == 0, then we are done because B > A
8  BEQ $t3, $t0, Label
9  # $t4 is C.
10 DADD $t4, $t2, $t1
11 DSUB $t4, $0, $t4
12 SD $t4, 2016($0)
13 Label:
```

4 instructions are executed dynamically, 2 memory-data references will be executed, and the code size in bytes is $7 * 4 = 28$.

Q4. Instruction Set Architecture

Consider the following fragment of C code:

```
if (A > B) {  
    C = - (A >> 16);  
}  
else{  
    C = A << 16;  
}
```

Consider the following constraints to answer this question.

- Assume that A, B, and C are 64-bit integers.
- Use instructions for 64-bit integers – e.g., use DADD (for 64-bit) instead of ADD (for 32-bit). Similarly, use LD/SD instead of LW/SW (for 32-bit). LD stands for *Load Double Word*.
- All data values are kept in memory (at addresses 2000, 2008, and 2016 for A, B, and C, respectively). Assume that the hex values in the memory addresses 2000, 2008, and 2016 are:

- Address 2000: 0x000012340000FEDC
- Address 2008: 0x0000FEDC00001234
- Address 2016: 0xFEDCBA9876543210

Memory addresses are given in decimal values for simplicity.

- Use BNE or BEQ instruction for the branching. (Hint: See SLT (Set-on-Less-Than) instruction. For MIPS references, check the course homepage.)

In the next page, write your answers for the following questions:

- Write the code in MIPS assembly.
- How many instructions are executed dynamically?
- How many memory-data references will be executed?
- What is the code size in bytes? (Count only the bytes for instructions.)
- Write the value (in hex) of memory address 2016 after the code execution?

Example 10.4

Let's start with the code.

```

1  LD $t1, 2000($0)
2  LD $t2, 2008($0)
3  SLT $t3, $t2, $t1
4  BEQ $t3, $0, ELSE
5  DSLR $t4, $t1, 16
6  DSUB $t4, $0, $t4
7  SD $t4, 2016($0)
8  J ENDIF
9  ELSE:
10 DSLL $t4, $t1, 16
11 SD $t4, 2016($0)
12 ENDIF:

```

I don't understand the dynamic instructions parts

There are 10 instructions so $10 * 4 = 40$ bytes.

The hex value of memory address 2016 after code execution is

0x12340000FEDC0000

Q5. Instruction Set Architecture

Consider the following fragment of C code:

```

for (i = 100; i > 0; i--){
    A[i] = B[i] + C;
}

```

Assume that A and B are arrays of 64-bit integers, and C and i are 64-bit integers. Also, assume that all data values and their addresses are kept in memory (at addresses 1000, 3000, 5000, and 7000 for A, B, C, and i, respectively) except when they are operated on. *Assume that values in registers are lost between iterations of the loop – i.e., if you load the value of i into a register in an iteration, do not assume that the register keeps the value in the next iteration.*

Write the code for MIPS. How many instructions are required dynamically? How many memory-data references will be executed? What is the code size in bytes?

Note that i contains an index value. To compute the actual address of an array element, it is necessary to multiply 8 to i and it can be easily done by using the DSLR instruction.

(a) Complete the MIPS assembly code given below.

```

1  DADDI $t1, $0, 100 ; set i = 100
2  SD $t1, 7000($0) ; store i
3  LOOP:
4  LD $t3, 7000($0) ; load i
5  # Check exit condition
6  SLT $t4, $t3, $0 ; if i < 0 then 1 else 0
7  BEQ $t4, $0, ENDLLOOP ; if i == 0 go to ENDLLOOP
8  SLL $t4, $t4, 3 # Multiply by 8
9  DADDI $t5, $t4, 5000 ; Calculate address of B[i]
10 LD $t6, 0($t5) ; Load B[i]
11 LD $t1, 5000($0) ; Load C

```

```

12  DADD $t2, $t6, $t1 ; $t2 = $ B[i] + C
13  DADDI $t5, $t4, 1000 ; Calculate address of A[i]
14  SD $t2, 0($t5) ; Store B[i] + C at address of A[i]
15  DADDI $t3, $t3, -1 ; Decrement i
16  SD $t3, 7000($0) ; Store new i
17  J LOOP
18  ENDIF:

```

(b) How many instructions are executed dynamically?

$$3 \times 101 + 10 \times 100 + 2 \times 1 = 1305$$

(b) How many memory-data references will be executed?

$$1 + 101 + 4 \times 100 = 502$$

(b) What is the code size in bytes?

$$15 \times 4 = 60 \text{ bytes}$$

Q6. Availability

If a company has 10,000 computers, each with a MTTF of 35 days, and it experiences catastrophic failure only if 1/3 of the computers fail, what is the MTTF for the system? (Assume that computers are not repaired.)

Note that if for one computer, if $MTTF = 35 \implies FIT = \frac{1}{35}$. For 10,000 computers, this means $FIT = 10000 \times \frac{1}{35} = 10000/35$ which then implies that $MTTF = 35/10000$. This can be interpreted as 1 machine would fail every 0.00035 days. The farm will crash every $10000/3 \times 35/10000 = 35/3 \approx 11.67$

Q7. Speedup

Assume that we make an enhancement to a computer that improves some mode of execution by a factor of 10. Enhanced mode is used 50% of the time, measured as a percentage of the execution time when the enhanced mode is in use. Recall that Amdahl's law depends on the fraction of the original, unenhanced execution time that could make use of enhanced mode. Thus, we cannot directly use this 50% measurement to compute speedup with Amdahl's law. Show your work for each question.

(a) What is the speedup we have obtained from fast mode?

$$\text{Speedup} = 0.5n + 0.5 \times 10 \times n = 5.5n$$

What percentage of the original execution time has been converted to fast mode?

$$1 - x = x/10 \implies x = \frac{10}{11}$$

Q8. Assemble Instruction

Write the hexadecimal representation of the following `SRL` (Shift Right Logical) instruction after it is assembled. The instruction is to right-shift a register value by the shift amount and to place the result in the destination register.

- This instruction is a R-type instruction.
- The *opcode* for `SRL` is 0 and the function code is 2.
- In the instruction below, the number after `R` represents the register number.
- Use `0x0` for the `rs` field value.
- You must give the values of the fields explicitly.
- The final hexadecimal representation must include all digits – i.e., do NOT ignore leading zeros. Also, use `0x` to represent it is a hexadecimal number.

`SRL R10, R4, 20`

Note that R-type instructions are of the following form: `opcode, rs, rt, rd, shamt, funct`. In binary this is

000000 00000 00100 01010 10100 000010

Taking every 4 bits yields

0000 0000 0000 0100 0101 0101 0000 0010

And in Hex this is

`0x00045502`

§10.2 Practice Set 2